

Dimensional Manipulation

Kureishi Shivanand [2025-09-16]

Abstract

As AI systems continue to evolve, their potential applicability should tend towards the same. Therefore, there's a need (or requirement) for the general (mainly non-STEM) public to gain an intuition of how these systems operate and process requests in deriving a solution. Since probability is the foundation of any statistical exercise, it's understandable why it remains at the forefront of any analysis. As such, these intelligent systems are inherently non-deterministic in their reproduction. As these systems account for more data, for which each has an increasing number of attributes to account for, their analytical complexity increases exponentially. However, it is only through this complex process that a useful level of "artificial" intelligence may be generated and displayed. One part of the analytical process is in formatting a "brain" to understand and process data. As this "brain" intends to replicate "true" intelligence (referenced on the human response through statistical means for this purpose), it is trained on human-referenced data and their usual responses (or at least until it passes a threshold of substantial understanding). The aim is through a statistical means, the system would arrive at a "correct" or "acceptable" answer, without the analytical or calculative restrictions present in humans mainly due to fatigue or bias. The other part of the analytical process concerns the user or the one who interprets and applies the results retrieved from the system. Some of the main issues which arise at this point are the preciseness of interpretation and the means for which requirements are addressed relative to the method which such system is subsequently accessed and queried. In traditional software engineering terms, this addresses the *validity* of solution, while the prior part addresses the *verification*. The paper will attempt to present solutions which may relieve some of the issues present at this stage. The use of dimensional manipulation may be employed for the purposes of investigating the data on a fundamental scale (as a utility which operates deeper than traditional feature engineering), as well as creating an efficient pipeline for the intelligent systems to recognize and extract patterns from. The basis of dimensional manipulation is formatted to be highly adaptive to the data for which it pertains to. For this reason, there can be no "absolute" formulation in which it is to be conducted. With that said, there are currently guidelines and proven methods in place which achieve substantial results. The implementation presented throughout this paper is meant to aid in a similar endeavour. That is, to provide an alternate perspective for thinking and approaching problems. The code or steps required to do so are not especially revolutionary; however, seldomly conducted as they are limited to an established convention, which itself is tied to certain domains. While there are some technical terms and formulations present, this paper is not meant to be highly technical, as its intended audience are aimed towards those who are not accustomed to designing and developing such systems. With that said, everyone may benefit from the concepts as it presents

methods in which traditional processes may be utilized in non-traditional domains. As such, once again, the encompassing purpose of this paper is to attempt to alter the mindset used in the analysis of data, and consequently, artificial intelligent systems. Rather than the various methods used to extract insights, if the fundamental source is altered (in this case, the analyzer), the results generated will not only be more distinguishable (higher cause-effect fidelity), but also be more cohesive in addressing the root issue (linking direct answers to a direct query). As mentioned, the code and implementation are not novel; however, the means through which such procedures can be employed may be. Hence, a sample procedural notebook and script are provided along with this paper (further details included in the *Proof of Concept (Possible Implementation)* section). This was made in the general case (in utilizing no specific dataset) in order to allow such a workflow to be applicable to a wider range of domains. However, as detailed in the notebook, the randomly generated data can be substituted for any data of a similar form (or any data presented numerically). Since this paper isn't meant to inform a new method, but a perspective on procedure, this should serve to exhibit its possibilities.

Motivation

The reason for seeking exploration within the topic of “dimensionality” lies in how and what ‘things’ can be interpreted. What information can be utilized from the observations we (humans) make through our senses. Furthermore, how accurate and reliable exactly are these biological sensors? Anyone with an academic background in traditional STEM, should have an intuitive sense of what the term “dimension” indicates. While the term can take on similar variants; it more or less relates to the scale in which the tangible, definable features of a subject of interest can occupy. This scale is made to be relative to the scope in which such features are measured. The most obvious means in which “dimensionality” is utilized is during the observation of physical phenomena. The specific aspects of an observable define the category in which such an observable is placed. These aspects each define at least one dimension in which the observable occupies. Such dimensions may be manipulated against each other to fulfill some purpose during an analytical process. For those familiar with standard Machine Learning concepts, “dimensions” define the capacity in which tangible information can be extracted. More on this will follow in later sections. For now, it's sufficient to comprehend that the quality of patterns extracted from data are contained within the features which represents them. This is essentially the main purpose of authoring this paper, in which further attempts will be provided to demonstrate the consequences of such an interpretation.

It's important to understand that dimensions are utilized whenever a subject is measured; regardless if the user realizes or not. Therefore, every decision which results in a tangible outcome is taken based on measurements across relevant dimensions. Consequently, a decision made based on no dimensions (different from the 0th dimension) is a decision made with no tangible evidence; thus, holds no merit. Throughout this paper, a sub-objective will be to provide insights into a perspective in which the world is constantly “measured”. This should prove useful for the reader as human society becomes more intertwined with AI resources. The main point in this regard, is the ‘scope’ in which to apply the discretization of an applicable scale, in order to classify a relevant dimension. In simpler terms, a miniature subject should be measured, then evaluated, on

a miniature scale, while a gigantic subject should correspondingly be measured on its own relative scale. The majority of issues which arise during misclassification are mostly due to incorporating an inappropriate scale when measuring a subject of interest.

One of the most well-known applications where this is relevant (at least for those from STEM) are the concepts pertaining to the “Classical” and “Modern” representations of Physics. First off, as a fundamental notion, it should be noted that Physics can’t be learned; only its various representations which describe the relevant phenomena. Such representations are derived from applying a relevant scale in which to use mathematics to conduct generalizations on computations. The generalization being in this particular case, the designation of “Classical” based on the subject of study occupying a specific scale. Modern physics conveys a wholistic understanding at boundary cases, in which the “Classical” regime is used to generalize calculations at particular-relevant scales; wherein such scales are expressed through the dimensionality of the subject.

This topic will be explored in greater detail within the sections that follow; however, it’s sufficient to understand that dimensions can be manipulated through factorization and aggregation, which in turn gives rise to the relevant aspects which are contained in such features. This is another important point this paper aims to convey. Just as all finite numbers contain factors (even primes contain at least two factors), it should therefore follow that the dimensions which represents such numbers should also carry their own relevant, factorized representations. Furthermore, as one gains greater insight into the composability of an entity once it’s deconstructed and examined, a similar situation should be sustained in terms of the dimension of such an entity occupies. The difference being, as it’s the representation, the “generalization” of the entity itself is that which is decomposed for the purpose of analyzation.

While the topic of this paper is pertaining to “dimensions”, it more so is intended to alter the reader’s perspective of they analyze the world, if required and possible. In the era of AI integration, what can be more useful than learning how to integrate these technologies, while attaining a sense of how such models are most likely to arrive at certain results. Therefore, while this paper may be somewhat technical while explaining the particular mathematical applicability utilized in the methodology; it’s intended for the general audience whom plans to “work with” AI for the duration of their life’s tasks. With that in mind, a simple example will be provided for utilizing dimensionality for a “world perspective”; albeit perhaps a bit extreme. Pertaining to the task of classifying human settlements; their primary environment (where one’s modern ancestors most likely originated from) may be inferred based on their physical attributes (i.e. skin colour, hair texture, height, etc.). This is a high-level view; hence, contains a high level of dimensions. Investigating deeper will consequently reduce the dimensions on the observations, generating a more modular insight. Here, regionality may start to be inferred, as well as migratory patterns (continent → region). At this level, the features are discerned to a greater extent as required for the classification; for example, the specific melanin contained in the hair and skin, skin condition, facial features, skeletal structure, etc. An important observation should be highlighted, although perhaps obvious: “Physical classification pertain to only physical features”. This statement should be obvious; however, it seems to be easily forgotten, if not known, for most of human society.

Within a broader outlook, the utilization of dimensionality manipulation has the potential to be utilized for a wide range of applications. Primarily, it allows for fundamental manipulation when designing architectures intended for multi-modal I/O applications. For example, in the case where a specification requires information extracted from images to be formatted into such a

representation that is optimal for speech-based applications. Once the representation can be independent from a particular scale, it will be unrestricted in respect to a particular modality. Thus, the aim of such an application can be defined as to train a system which takes in an image; however, it is processed as an audio clip downstream. This paper explores the situation where data has the capacity to be represented as “pure information”, in an attempt to allow for the seamless transfer of knowledge. This ambition is akin to “knowing” what an apple refers to, rather than first determining what to visualize when the phrases “apple” or “りんご” are uttered. While LLMs attempt to abstract away the concept of language; as its very name suggests, the concept of ‘language’ itself is built-in, leading to inherent natural language analysis and interpretation.

Intuition

Perhaps the most important tools when conducting professional work in this era are computers (at least for those in STEM). These may include cellphones, laptops, calculators, etc.; essentially anything that performs computation. Consequently, to classify these entities as “tools”, requires the user to comprehend how such a tool arrives from one stage to the other. This isn’t to detail how the circuitry accomplishes the computation; it’s sufficient that an understanding can be formed (within a relevant tolerance) of how to get to point B from point A. Hence, reproducibility is the key when utilizing tools; the user expects a certain format of the results to appear in its presentation. Therefore, in the case of AI, the same should apply if a similar supporting relationship is desired. In order to use AI aids to a workable, long-term extent, the user should have an idea of how these “tools” arrive at a particular solution. Thus, while this paper may prove helpful for those also in STEM, its main demographic is towards those with a lenient technical background. As generative AI tools have become mainstream at the time of authoring this paper, their utilization have also been incorporated into almost all working industries. Therefore, their users should not only know how to properly use them, but also have a feel of how such tools tend to solve tangible, relevant problems. Note that AI applications are used as an example here since they are believed to be a “new technology” which is relevant to most of society; however, the concepts presented here relating to ‘dimensionality’ may very well prove useful in other areas as well. The concepts presented allow for a re-evaluation of how one may perceive the world and arrives at decisions. Although such concepts are “presented”, the essence of them are by no means “novel”; therefore, they should be relatively simple for one to grasp. This paper aims to have the ‘non-technical’ user be able to confidently navigate an increasingly ‘technical’ world.

Since the following sections will contain the actual formulations of the concepts, this section will simply aim to provide the reader with a feel of how dimensions should be interpreted. To start, it’s useful to have the following geometric imagery in mind. The simplest, definite dimension that’ll be accounted for is the 0th dimension, which is represented by a single point in space. The next step from a ‘single’ point is to have ‘multiple’ points. This in turn leads to a single line, which is taken to be the 1st dimension. A line by definition is many points along a pre-defined direction (axis). This process continues, in that multiple lines lead to a single plane, and so forth. From here, it can be seen that representations of a certain dimension builds upon each other to form more complex expressions, which are of a higher dimensionality. These escalations are necessary when the data to be represented becomes too complex to model through rudimentary means. Consequently, for the very fact that these aggregated representations are formed from many representations, it follows that such representations can be broken down into smaller,

modular pieces themselves. In other words, the multiplicand and multipliers that form a product are by definition its factors. With this in mind, one can see how the process can be reversed to generate one from the other. The latent space in an autoencoder architecture shares similar co-relationships (Bergmann, 2025). In brief, simple terms, the encoder deconstructs the representation of some input data, while the decoder attempts to construct a similar representation of such data. The area of transition is the latent space (which holds the “latent/inherent” representation). While these processes are quite similar, there are subtle differences that exist, which ultimately was a primary reason for writing this paper. The main difference being, the latent space takes a representation of a larger number of dimensions and attempts to shrink it while retaining the important, essential information, which constitutes such representation. However, these are not always linear representations, in that they don’t consist of reproducible factors at every instance of the process. And even if they do, the subset of retainable information is decreased due to the very nature of the process. This stems from the fact that the latent representation is derived from a probability distribution encompassing the “essentially-important” features of such data. As this isn’t the point of this paper, not much more should be mentioned on this matter. This was only included to demonstrate why the reconstruction (“decoder”) process contains inevitable variability. The proposition essentially aims to reduce such variability to allow the entire process to be more traceable on a fundamental basis.

Essentially, the proposed method aims to account for multiple trajectories of reduced-dimensional representations, in order to gain a wholistic view of the investigative data. There exist multiple well-established methods to accomplish this since the methodology itself is not novel. In this instance, a simple implementation which utilizes a combination of decision trees and random forest architectures will be conducted. The specific details of the implementation will be highlighted in later sections. With that said, as this is merely used for a proof-of-concept and not the main purpose of the paper, its specific concepts will only be explained briefly. The main purpose is to attempt to present a different perspective of thinking, analyzing and understanding data, in a pursuit to allow for a “more comfortable” integration with AI.

As alluded to prior, the dimensionality of a representation is directly linked to its capacity to convey information. Generally, the more dimensions a representation contains, the higher its potential ‘information capacity’. It is also important to remember that the scope of investigation can provide insight into the representation, which in turn may lead to discovering more of its constituents. An example of this is demonstrated from how a standard 2D line in the macro world ‘looks’ like a 3D rope in the micro world, or perhaps even a plane on smaller scales. Hence, the level of scrutiny and discerning ability applied to the informative analysis corresponds to the information which can be extracted and utilized. In other words, the ability to manipulate its dimensionality corresponds directly to the insights which can be formed from data representations. Once the scope of investigation hits the optimal maximum, wherein the scale is at the minimal, the information is said to be represented in its most modular form. In most cases, these modular forms consist of binary expressions, which are taken to be 1D (requires one variable to be adequately represented). While it may prove quite difficult to attain this representation, it was conceived to express the fundamental “yes/no” response. This decision was made for its highly-consistent traceability and flexibility in representing “activated states”. Furthermore, within literature, “yes/no” can be taken as the most basic response. As this is the ‘ideal’, it should be treated as the “limit in which to approach” and not a requirement. For instance, if the level of “vagueness” in the question doesn’t allow for a sensible binary response, two or more dimensions will be necessary to account for the different aspects in which to respond to. Another way of interpreting the representative formation is to have each modular representation of a dimension account for a “yes/no” aspect of

the data to the ‘best of its ability’. For those familiar with the naïve design architecture of CNNs, this process is similar to the utilization a kernel function, windowing along various portions of the data (image), aggregating the results, then repeating until the entire data is consumed. The kernel acts to “increase the scope” in order to focus on a specific aspect of the data; thus, allowing for more direct analysis.

To reiterate, dimensions define the capacity for which data is able to represent its information. Within a geometric or graphical sense, one can think of a set of axes as representations for the attributes of such data. An axis can be defined as long as its representative attribute is independent (not inherently dependent upon) of any other present attributes. This requirement is important in ensuring modularity of expression and traceability of factors. This isn’t to say that the axes can’t correspond to each other, in that they may be based upon the same measurement scale. However, they shouldn’t correlate among each other leading to a situation where the same attribute is described along a particular dimension. The more technical details will be discussed in great length later on. For now, it’s sufficient to just understand that in order to gauge the amount a particular attribute influences a feature, the ‘independence’ requirement is crucial and must be upheld. The further a feature is probed to investigate its associated factors, the more essential this requirement becomes. As the dimensionality of the factors reduce, in addition to attaining a higher number of such factors, the combinations to which they can aggregate and disintegrate increases exponentially. Therefore, to ensure traceability and modular representation, this ‘independence’ requirement is paramount.

Conceptualization

Before the formal processes are laid out, it’ll be useful to define some terminology and concepts that will be utilized later on. Some of these concepts have already been discussed, but will now be given official names that will persist throughout this paper. ‘Dimensionality Reduction’ relates to the process that generates factors, which arise once a representation is decomposed into its constituent components. As this process can occur multiple times, the resultant expression is related to the previous expression on a one-to-one basis. In other words, the n^{th} expression is the reduced expression of the $(n - 1)^{th}$ expression. This distinction is important due to the variability or various trajectories the process of reduction can take, based on the what the target of investigation is. The reverse process can be taken to be the aggregation or combination of such factors. This process will be referred to as ‘Dimensionality Aggregation’. Both processes together are collectively known as ‘Dimensionality Modification’. By the definition of factorization, the dimension of a representation is proportional to the number of factors it can contain. Hence, as additional factors are aggregated together, the dimensionality increases. Note, the dimensionality is not linearly related to the information a representation contains, since different factors may be of a more complex nature than others. Furthermore, certain factors may have more influence on the overall representation than others. In other words, certain components contribute more heavily to the data in which its relevant representation is attempting to model. In terms of traceability, these components will have a more direct relationship on the analyzed feature, thus having greater influence on the shape of its overall distribution.

The basic methodology behind ‘Dimensionality Reduction’ is as follows:

- 1) Generate an expression which describes a single feature under analysis.

- 2) Conduct initial factorization on said feature to determine factors of its original expression.
- 3) The resultant expression is further reduced until the limit of modularity is attained, while detailing the relevant influence of the factors from within.
- 4) The reduced expression provides insight into the traceability and information capacity (discussed formally in later sections) through the resultant features produced and their dimensionality.

If the procedure is conducted as expected, the resultant expression through using this methodology not only gives rise to an expression that is simpler to model, but also contains a higher level of explainability compared to its primitive counterpart. In order to accomplish this, only a negligible (in which the threshold is pre-defined by the user) amount of information is lost. As such, the generalization of such data should be maintained, in that the overall information presented can still be ascertained.

With all that said, how exactly is the dimensionality of features determined? The primary tactic employed lies in the discernibility for which the investigation takes place in. The perception and level in which to conduct an investigation determines the level in which to report relationships; hence, presenting the relevant factors of a particular feature. Afterwards, the dimensionality can be determined from these relationships, among the relevant features. It should be noted, as dimensionality is inherently based upon a feature and its present relationships, it can be said to have a 'relative' relationship. That being the case, the relationships are based upon a "leveled tier". Essentially, as mentioned before, certain features might influence some features more than others; thus, forming a 'closer' relationship. Furthermore, through probing a feature for further insights, more dimensions may arise, which in turn may lead to more features that also have their own relative relationships. All these ideas will be elaborated on later in the paper. This is another reason why the modularity requirement is of such importance. In order to ensure a higher percentage of definite (reproducible) results, the trace in which features are formed needs to have the capacity to be recorded in great detail. If modularity isn't accounted for, there is a higher risk of error in determining an appropriate representative expression, which leads to a further amount of non-deterministic results. Hence, modularity isn't only for a cleaner representation, it is also fundamental in tracing the leading factors of a feature which defines its behaviour.

The act of modelling is inherently an iterative process. The factors which define the distribution of a particular feature are analyzed then presented in such a way that attempts to reproduce the behaviour of said feature. In other words, the factors constituent the behaviour which define their source feature. As such, the relevant factors of a particular feature are its 'constituents'. Furthermore, the detail which a representative model contains can be modified by adding or subtracting its relevant constituents; thus, manipulating the resultant dimensionality. The complexity arises with the aim of striking a balance between overfitting and underfitting. Overfitting is more likely to occur with representations of a high dimensionality, while underfitting for a lower dimensionality. This is due to the presented utility of 'dimensionality'. Higher dimensions are required to capture the behaviour in highly complex data. Thus, "more complex" data can be interpreted as the level of variance within that needs to be modelled through higher-sophisticated means, in order to capture the behaviour of data to a respectable degree. Due to compute resource and time restrictions, an excess number of dimensions shouldn't be employed if they aren't needed. Consequently, the other end of the spectrum should also be accounted for: in the case of underfitting representations. This particular situation occurs when the behaviour to be understood requires too few dimensions, due to the limited complexity of the data. One such cause of limited

complexity may be due to the reduced variability in data behaviour; thus, the moniker of “other end of the spectrum”. Therefore, the model will be biased towards a particular way of thinking represented by the available limited data. In such cases, the model is said to be afflicted with ‘high bias’. With this in mind, it can be seen why determining an optimal number of dimensions to exploit is detrimental in attaining a high-performant representation of data. Furthermore, as factorization may be utilized in manipulating the number of representative dimensions, it is also a means used in examining data through various perspectives.

To reiterate once again, the purpose of this paper is not to present novel ideas or implementations. The purpose is to demonstrate how one may efficiently utilize resources to gain insights of interest. By accessing relevant aspects of data, which in turn is likely to raise the effectiveness of the proposed model architecture, its capacity is heightened in order to capture trends more accurately and generally. In fewer words, it presents a methodology to gain insights into data through utilizing a dimension-based approach. The methodology proposed is intentionally designed to use implementation techniques that should be familiar (at least to those in the machine learning sector). In this way, the practical utility of dimensionality will be segregated from its implementation details. Additionally, it’s also an ambition that this methodology can be integrated in one’s current analysis workflow, and not invent new technologies for the purposes of incorporating it. First principles methods are utilized to also allow one to visualize how current workflows may be employed for non-traditional sectors.

Brief descriptions will be provided here, which only details the necessary components of each sub-architecture of the procedure. For the purposes of this section, “design pattern” and “design architecture” can be taken to refer to the same concept. As there is a plethora of information on these already present within the literature, it’s deemed that one can provide their own clarification as required. The first fundamental architecture which is utilized is ‘Principal Component Analysis’ (PCA). The fundamental purpose of PCA is to reduce a high dimensional feature set into a lower representation, which still retains an adequate amount of information from the original set. The specific threshold of ‘adequate’ is to be set by the implementer; however, a robust benchmark to go with is to retain ~ 98% of reproducible behaviour. This would approximately cover the distribution between the 2nd and 3rd standard deviations of a standard Gaussian distribution, which should be sufficient for the task of ‘understanding’. However, to reiterate, this metric is set by the implementer and may be subject to vast alteration depending on the particular circumstances. The main reason this technique is usually implemented is to allow for training to run more efficiently, in that the model would be steered towards learning the characteristic behaviour rather than resultant instances. Since the feature set is essentially ‘summarized’ in a way, the relevant patterns should become more easily extractable, for which the model should pick up more efficiently; since as a result, there are fewer variables to keep track of. This is used as a fundamental building block to demonstrate the proposed methodology due to its inherent capability in dimensionality manipulation. The second fundamental architecture which is utilized is the ‘Decision Trees’ mechanism. The primary purpose for this decision is to attempt to account for the variability in aspects which influence a particular feature’s behaviour. As behaviour may require various aspects, as well as different levels/intensity of investigation per individual aspect, in order to gain an adequate understanding, receiving various insights from different perspectives (simulated by the potential paths traversable within a decision tree) should prove beneficial to one’s analysis by providing an encompassing outlook of said behaviour. For those familiar with it, the idea is similar to the ‘dendrogram’ architecture, where decisions are made on a cause-and-effect basis, which in turn designates a path to be more probable than another. Through analyzing features from different perspectives, various aspects are measured and accumulated in

determining the intricacies which define the investigative feature. The third and last fundamental architecture which is utilized is the ‘Random Forest’ design. This is implemented as a means of accumulating and discerning insights from the various aspects attained from the decision trees during the previous stage. The idea here is simply to use a feature’s discerning aspects to differentiate its similarities and differences from one another; thus, forming corresponding relationships amongst themselves. An analogy is how all humans are ‘human’; however, it’s their intricate personality (which define likes/dislikes) which has the greatest influence on the degree of relationships formed among them. The implementation and interconnection details among the architectures will be discussed in-depth during later sections. This was only to give a brief description of the 3 fundamental building blocks within the methodology, as well as the decision towards their utilization. Once the aggregation and discerning process is completed, the results are passed to an analysis machine (either a computerized model or human “expert”) to extract relevant insights concerning the feature’s behaviour, with the goal of meeting a certain objective or requirement.

Foundational Formalization (Stage 1)

This section expands on the implementation details presented prior; as such, some concepts will be discussed once again, albeit with more concrete examples and mathematical descriptions. To start, a formal definition of ‘dimensionality’ will be provided, which will be sustained for the remainder of this paper. ‘Dimensionality’ will henceforth refer to the specific dimensional level set on a system of interest. As such, dimensionality can only take on the values 0 and positive integers. Formally, this is expressed as $dim \in \{0, \mathbb{N}\}$. Since 0 may or may not be agreed upon as part of the “Natural Numbers” category, it was stated explicitly as part of the set. Hence, no negative or fractional dimensions should be taken into consideration for the scope of this paper. This isn’t to say they don’t exist; however, to understand them would require delving into “Probabilistic Mechanics” and other relevant material, which is out of scope and not required to convey the point of this paper. In other words, only discretized-defined dimensions will be discussed. The following presents a mapping between dimensional levels and their geometric representations for the first four dimensional levels, as these should be relatively simple to visualize:

Dimensional Level	Algebraic Representation	Geometric (Spatial) Representation
0 th	constant	point
1 st	x	line
2 nd	$x \cdot x = x^2$	plane
3 rd	$x \cdot x \cdot x = x^3$	space
4 th	$x \cdot x \cdot x \cdot x = x^4$	volume

Table 1.

The algebraic representations are explicitly written out to indicate the cumulative nature of dimensions. The geometric representation is interpreted in a spatial perspective for ease of visualization; however, it can just as easily occupy a temporal or any other “realm”. As long as all the data which is to be represented is measured on the same scale, the user is free to manipulate the representation to suit their specific needs at hand. This too is a fundamental application of dimensional manipulation. A transformation of the corresponding basis vectors, which define a particular representation, is essentially its span (more on this later). The ‘basis’ vectors can be

manipulated with respect to scale and scope to suit various, relatable aspects of a particular feature. With regards to Table 1, if we let n represent the dimensionality of a feature of interest, then

$$\text{Information Capacity} = I_c = x^n \text{ (Eqn. 1)}$$

Eqn. 1 represents the ‘explainability’ of one particular feature. This quantity provides an indication as to the amount of tangible information which can be contained within a feature. It’s important to note this quantity varies depending on how thorough or discerning the investigation of said feature is in determining its constituent aspects. Specifically, x represents the elements used to describe a single dimensionality (of one feature), and n describes the capacity of such dimensionality with regards to a single representation of its data.

The elements, x , themselves are based on a tangible, verifiable scale. These may also be combined and decomposed to represent new or different information. Consequently, this may lead to the feature transforming to a representation with more or less dimensions. Due to the scale of highly-complex features, various attempts are applied to decompose them into more modular ones, with the goal of extracting information that is particularly relevant in aiding the investigation at hand.

Corresponding axes can be expressed using the same ‘ x ’, as long as their elements describe similar, relevant attributes on the ‘same’ scale. A common example of this is the vastly-occupying “3D” spatial scale $\rightarrow x \cdot x \cdot x = x^3$, which can adequately be described using 3 axes, leading to such data being represented within the 3rd dimensionality.

A concrete example of combining dimensions is given by the “Spacetime” or “World Map” representation. Here, let x describe spatial-based attributes and y describe time-based attributes. Hence, the resulting description yields a $(x \cdot x \cdot x) \cdot y = x^3y^1$ representation. Note: When one dimension is represented, it’s possible to represent it without an explicit exponent, since these are expressions used to describe information within the geometric sense. Thus, $x^3y^1 = x^3y$.

It’s important that each dimension only correspond to a single aspect. For the purposes of this text, the terms “aspects” and “attributes” can be taken to be used interchangeably, even though their specific linguistics may differ. Therefore, it’s useful to think of features as containing multiple aspects; thus, containing multiple features. In this way, a ‘mega’-feature may be taken to contain many other sub features, which are its own aspects or attributes of defining behaviour.

Demonstrative Example

Let’s assume that a feature exists that describes the area of a house, while the actual quantity of interest is the perimeter of its occupying property. Through an initial investigation, it’s discovered that the area is rectangular-based. Hence, the feature ($A = \text{area}$) contains 2 aspects ($l = \text{length}, w = \text{width}$). Consequently, this single feature contains 2 dimensions ($n = 2$). By its geometric definition, this expression would be represented by a plane, which in turn is described algebraically by a standard matrix. If l is indexed column-wise and w is indexed row-wise, the resulting description of the feature is:

$$A = \begin{bmatrix} w_0 l_0 & w_0 l_1 & \cdots & w_0 l_j \\ w_1 l_0 & w_1 l_1 & \cdots & w_1 l_j \\ \vdots & \vdots & \ddots & \vdots \\ w_i l_0 & w_i l_1 & \cdots & w_i l_j \end{bmatrix}$$

Here, i represents the number of elements used to describe the width, while j relates to the length. For instance, take the length attribute. The number of representative elements, x , is determined by examining which part of the attribute is relevant to the particular investigation. In terms of determining the property perimeter, information which may be relevant are the usability of the land, obstacles and obstructions, how much of the land is fenced, etc. Assuming the noted information contains all which is required (in practice, there should be much more), then $x = 3$. Thus, from [Eqn. 1], the information contained within l is

$$(I_C)_l = x^n = 3^1 = 3$$

In this case, w can be assumed to be described in the same respect. Hence,

$$(I_C)_w = x^n = 3^1 = 3$$

Combining the information capacities of both attributes results in $(I_C)_A$

$$(I_C)_A = (I_C)_l \cdot (I_C)_w = 3 \cdot 3 = 9$$

Notice this is the same result for if Eqn. 1 was used, with setting $n = 2$ as mentioned before:

$$(I_C)_A = x^n = x^{n_l + n_w} = 3^{1+1} = 3^2 = 9$$

Here, it can be seen that two perspectives, one with explicitly compounding information capacities and another with accumulating dimensionality, yields the same results. This is in fact what one should be expect, since they arise from the same source (Eqn. 1); however, expressed in different terms through different perspectives.

Through utilizing trends/patterns within data, as well as domain knowledge, the feature of interest can be transformed into one such that predictability for the quantity of interest can be attained. Extracting relevant attributes which greatly influences a feature's behaviour, then formatting them into a compact representation, allows predictive analysis to be conducted more efficiently, in comparison to with a greater number of irrelevant features. This is the primary reason in which the common technique of PCA is conducted (exact implementation details in later sections). Referring back to the working example, utilizing domain knowledge of rectangular-based formations, yields:

$$Area = A = lw \rightarrow 2(l + w) = P = Perimeter$$

Therefore, if A can be tracked, the proportionality; hence, the qualitative representation, of P can also be determined. In other words, a trend can be formatted for P .

To attain a quantitative representation of P , further data, investigation, and/or techniques would be required. The cause lies in the level of investigation being of a greater degree when compared to its relevant, qualitative representations. This is usually the case where the categorical bins of possible responses (traceable results) are of a larger number. More concretely, continuous-numerical variables contain more possible responses than discrete-alphabetical variables. Therefore, in order to format continuous representations towards a similar level as discrete ones, a higher level of scrutiny needs to be implemented within the investigation. Accordingly, the dimensionality required to represent these continuous variables are larger than those used for discrete representations,

precisely due to the fact that such representations tend to require more information to distinguish them from one another. That being said, it may be possible to combine its dimensions in a way that such desired, interrogatory information is more easily extractable for analysis purposes (either computerized or human-based). One such solution is to utilize a modified version of PCA, which not only condenses dimensions for relevant information, but also takes into consideration the goal of investigation. This modified version may designate greater emphasis on particular “compression methods” over other ones, given the specific task(s) at hand.

As such, in order to attain a more automated, ML-based approach; firstly, a brief description of how PCA may be integrated will be provided here. As PCA contains many hyperparameters which can be fine-tuned, as well as various implementations, only the applicable and relevant concepts will be detailed here. As mentioned previously, the main utility of PCA is to essentially decompose many defined features; which can later be combined in such a way, which may prove to be more efficient for training. This new representation is often presented using a different scale and units. If certain extractable information of a feature can explain its variability (after normalization) to an acceptable degree [1], it can then be classified as its aspect. The idea is that the model will have to learn less features, along with providing less variables to keep track of during analysis. The ability of PCA to decompose features into a more concise, relevant format is precisely the aim of this stage of the methodology. Another way in which PCA may be thought of is through its ability to infer a relational heatmap conveying the influence of features on one another.

As alluded to above, the level of abstraction (yielding from their dimensionality) among features needs to be defined. If given in terms of a proportionality ($A \uparrow \rightarrow P \uparrow$), the task of attaining the target variable becomes much less complicated in comparison to the case where exact quantitative measures are required. This too can be explained using a dimensional perspective. The I_C would be higher in cases where more information is conveyed from quantitative values rather than qualitative instances. Therefore, the representative dimensions of such features would also be higher, leading to more data for the model to learn; hence, its heightened “complexity”. This notion was already alluded to prior. In addition, the level of abstraction required from the analysis determines how to formulate the transformed feature after PCA computes its composite aspects. Utilizing other features, or feature engineering new representations, may also prove to be a viable option in the case where an acceptable threshold of representation isn’t attained.

Algebraically, assume x_1 and x_2 are the dimensions of a particular feature of interest. Hence, their elements, which express these variables, define the dimensions on which the feature acts. For those familiar with linear algebra terminology, the feature of interest ‘spans’ x_1 and x_2 (occupying space defined by combinations of these two vectors). This concept relates to the ‘basis’ as mentioned before, where the vectors define the feature’s capacity for representation. Note, as mentioned above, the elements of these two aspects, by definition, can’t depend on each other (aids in maintaining a modular representation). To reiterate once again, the capacity in which these two aspects influence the feature of interest can be either determined by utilizing domain knowledge (human-in-the-loop) or PCA (autonomous). As previously insinuated, PCA may also be used to generate a feature-correlation heatmap. Keeping this in mind, x_1 and x_2 can be formally represented as:

$$x_1^1 = [x_{11}, x_{12}, \dots, x_{1m}]$$

$$x_2^1 = [x_{21}, x_{22}, \dots, x_{2m}]$$

Here, the subscripts of each variable indicate the index of a specific aspect with respect to a particular feature. In this case, there are two aspects; hence, two indices. The superscript is used to represent the dimensionality of each aspect, in correspondence to Eqn. 1 ($n = 1$ for both aspects). Note, the superscript indices start at 1 instead of 0 to use “natural-numbering” notation and to express the physical meaning behind dimensionality. The subscript indices were also made to follow this convention in order to stay consistent; however, these indices may just as easily be 0-based, wherein the number of aspects would be denoted based on $(a + 1)$, if a was the original subscript number. Each aspect contains m (sub-subscript) elements in their respective representation.

Dimensional Adhesion/Compounding/Composition

From the description above, suppose if x_2 provides more information than x_1 in relation to how the behaviour of the “source” feature of interest changes. In this case, the specific combination used to represent such a scenario, through utilizing the tensor product operation, can be expressed as:

$$x_2 \otimes x_1 = [x_{2_1}[x_{1_1}, x_{1_2}, \dots, x_{1_m}], x_{2_2}[x_{1_1}, x_{1_2}, \dots, x_{1_m}], \dots, x_{2_m}[x_{1_1}, x_{1_2}, \dots, x_{1_m}]] \text{ (Eqn. 2)}$$

Forming the expression in this manner allows for the weighted, influential nature of composite dimensions within the particular feature to be detailed. By the definition of tensor products, the resulting tensor would be contained within the description of

$$\mathbb{R}^m \otimes \mathbb{R}^m = \mathbb{R}^{m \cdot m} = \mathbb{R}^{m^2}$$

In this case, the two features are taken to contain the same number of elements. This is often the case when the two dimensions occupy the same relative scale and units. In other words, they correspond to the same ‘frame’ of extractable information of data. However, this may not always be the case, which is the reason why the number of elements (denoted by m) are stated explicitly in the expression.

Dimensional Decomposition/Factorization

Utilizing the above expression, it’s straightforward to define a reverse process. Assuming x_{1_1} and x_{1_2} are aspects of x_1 , the reverse process of Eqn. 2 is defined as:

$$x_{1_1} = x_1 \overline{\otimes} x_{1_2} = \{x_{1_1} \otimes x_{1_2}\} \overline{\otimes} x_{1_2} \text{ (Eqn. 3)}$$

In most cases, just as multiplication is usually simpler than its factorization counterpart, dimensional composition is much less time-consuming and straightforward than its decomposition counterpart. This is due to the many combinations that are possible to form a desired dimensional combination from a set of aspects. Hence, it follows from Eqn. 3 that the following axiom can be expressed:

$$I \otimes x_{1_2} \overline{\otimes} x_{1_2} = I \text{ (Eqn. 4)}$$

Here, I represents the identic-value within the specific representation. In the case of Eqn. 3, it can be taken to represent an m -sized vector of which each element is 1. In other words, I will be a vector that once scaled by a certain value, returns that same value. Since this specific process acts as the reversed operation of a tensor product, the symbol $\bar{\otimes}$ is used to signify a negation to the original operation as taking the “reverse tensor product”. In addition, this complexity increases as the dimensions required to represent a piece of information grows. A continuation of the “area-perimeter” example detailed above will be used to illustrate this point.

Suppose that $l = (a + 7), w = (a + 4)$; where a is some positive, real-numbered variable. Therefore, $A = lw = (a + 7)(a + 4) = a^2 + 11a + 28$. Note that since a is raised to the 2nd power, it requires 2 dimensions to be represented (highest ranked). However, the entire expression of A itself is segregated from its constituent aspects. Simply put, the pure expression A , is taken to be sub-independent on its constituents, since the expression itself may be formed from multiple combinations of different factors. For example, both (5, 4) and (10, 2) form products of 20; therefore, the product isn’t solely dependent or favours one set of factors over another. Thus, it can be seen how the above expression may also have multiple combinations of constituent components. Consequently, $P = 2(l + w) = 2l + 2w = 2(a + 7) + 2(a + 4) = 4a + 22$. Here, the expression for P is represented by 1 dimension, since the a in the expression is raised to the 1st power. This leads to a very important observation. The dimensionality of a particular data representation is determined by its composability, not its constituent aspects. In the case above, it’s the direct composability which affects the result, not its individual factors. Even though both A and P were determined from the same two aspects, the exact form caused two different dimensionalities to emerge. This is why dimensionality is more so based on the compounding/scaling effect of factors and their rank, rather than the specific factors themselves (for most cases). An analogy with respect to construction would be that the factors themselves are the building materials (bricks, cement, etc.), while the composability is the architecture of the building (floor plan, circuitry, etc.). Only certain factors would satisfy the composability requirements needed to form a particular feature; hence, the dependence can be said to lie in the composability of aspects. For A , 2 factors (each rank 1) were combined through multiplication to provide a product of dimension 2. However, in the case of P , the factors are added instead of multiplied, then scaled by a 0th dimension representation (constant). This seems like an obvious observation (which it should be); although, it is not usually explicitly stated. As it’s fundamental to the dimensionality perspective, it is crucial the user understands its implications.

The factors provide an indication on the possible combinations which may arise from a feature. Furthermore, they define the potential for other possible representations which may arise from a particular feature. As such, if new representations need to be formed from a specific feature to gain alternative insights, these factors would be the foundational “seeds” which they arise from. This again reinforces the dependence on composability; different features can be formed using the same factors depending on how they’re composed. As alluded to prior, the dimensions of factors (constituent aspects which are combined together through multiplication) form the overall dimensionality of the resultant representation. Therefore, $\dim[P] = \dim[2(l + w)] = 0 + 1 = 1$. However, in the case of P , since the dimension is lower than A , finding its composite aspects should prove slightly simpler. This isn’t always a beneficial consequence, as it limits the flexibility for various options down the line, when attempting to generate a “dimensionless” representation of data, since the aspects which can be used are restricted.

Traversing Variational Paths (Stage 2)

While the above design architecture accounts for manipulating present features to attain relevant insights, it's limited to the variety it can recognize and capture. In other words, the process detailed above operates on a single available feature, ignoring the others. Therefore, insights may be inhibited since they were only extracted from a single feature rather than the total available data. This may even be the case where significant dimensional manipulation was conducted on a particular feature; yet, it was still inhibited by its lack of information. As this is an inherent issue with the proposed procedure, potential remedies were issued as a means to alleviate it. With that said, it's not so much a remedy, as an attempt to make the methodology more robust and flexible in its implementation. As mentioned in an earlier section, the proposition is to account for a variety of possible trajectories (relevant aspects from different, present features), then accumulate them into a representation, with the aim of maximizing the extractable information attained from it. To accomplish this, the "decision trees" design and "random forest" architecture are utilized. As these are fairly common design architectures (similar to naïve PCA), only the points relevant to the proposed methodology will be discussed. For more information on all these, as well as PCA, refer to the internet or literature, as there are a plethora of information and various implementations actively available.

Most of the reasons for utilizing these two design architectures, along with the variant PCA implementation, can actually be understood/visualized from their namesakes. As mentioned before, if the resultant representation from stage 1 (PCA-based operation) is referred to as the "seed", then it can be assumed that a "tree" (more generally, plant) grows from such a seed. In the broader sense, a "seed" can simply refer to any representation from which information can be extracted. This may refer to a present feature or a representation which arose from dimensional manipulation, resulting in a constituent aspect or an aggregation of aspects. Regardless, both can be treated as 'features', since it's possible for dimensional manipulation to be performed on them (to a certain limit which will be discussed in a later section). In case it wasn't clear prior, any aspect which is derived from a feature, is itself a feature which further operations can be performed upon. Thus, a "branch" can be defined as the operation which was performed on one feature, from which resulted in another, new representation. Overall, each tree has a designated source-feature, which is present in the original data, while the "leaves" of such tree are the resultant representations attained from performing various dimensional manipulatory operations upon the source-feature. Additionally, the number of trees would be proportional to the number of original, relevant features where tangible information can be extracted. Note, this is a proportionality and not an exact correspondence, due to the possibility that there may exist features in the original data which may depend heavily on other features. As mentioned repeatedly before, this deep correspondence is unfavoured when forming modular representations, and also hinders the analysis by incorporating "redundant" information. With that said, most features are recorded and presented with modularity in mind with respect to the data as a whole; hence, this only accounts for a minute number of instances, if any. Therefore, the proportionality would still hold for all general cases. This was stated only as a cautionary during implementation in order to make the procedure robust against all conceivable anomalies. This essentially covers stage 2 of the methodology. To summarize, the main function of this stage is to perform the procedure highlighted in stage 1 on all original, relevant features, instead of only a single one.

Aggregating for Robust Insights (Stage 3)

Once representations are attained from the various trees, it's time to commence the final stage. This stage functions to collect all relevant representations with the purpose of analyzing, aggregating and averaging out the representations for a generalized interpretation. This is conducted using a standard random forest classifier architecture. This decision was made due to its simplicity, flexibility and how well-known it is within the machine learning community. As the point of this paper is not to introduce novel ideas and implementations; a demonstration of how to formulate a perspective in which to conduct analysis to gain deeper insights using well-known tools is presented. Through this method, production of results should be made faster, while the performance of such crucial analysis should persist for a longer period. Though this stage might not be technically difficult to understand, its implications are crucial to the fundamental utility behind dimensional manipulation.

First off, the scope in which the classifier bins results received from stage 2 correspond directly to the level of scope which was designated in stage 1. Consequently, the distinguishability of results is also designated during stage 1. Scope has been mentioned multiple times throughout this paper as its paramount in the interconnectivity between the processes. The scope essentially defines the level in which to interpret results. Therefore, the scale at which stage 1 produces constituent aspects defines the scope of resultant insights. The granularity of these insights determines the capacity in which analysis can be conducted to acquire a certain depth of results. An example for which the scope affects analysis can be determining the difference between an apple and orange versus the difference between a macintosh or red delicious (variety of apples). Since the I/O of each stage can be thought of a systematic pipeline, the decision (hence, "decision" trees) made in the beginning of the procedure propagates throughout until the final results (representations) are attained; wherein, conclusions are drawn from.

Note that any aggregator can be placed in stage 3 in order to accomplish a specific purpose. However, "random forest" was chosen for its ease-of-integration with "decision trees", in addition to its simplicity of extracting results during analysis. The operation performed after aggregation will henceforth be referred to as the "analytic operation". The analytic operation can be designated to perform other tasks from averaging results. Essentially, any kind of optimization tasks may be performed to meet pre-defined specifications required for adequate extraction. This may include maximization, minimization, averaging, summation, ranking, etc. It should be noted however that the precision in which the operation produces results adheres to the scope of investigation as well.

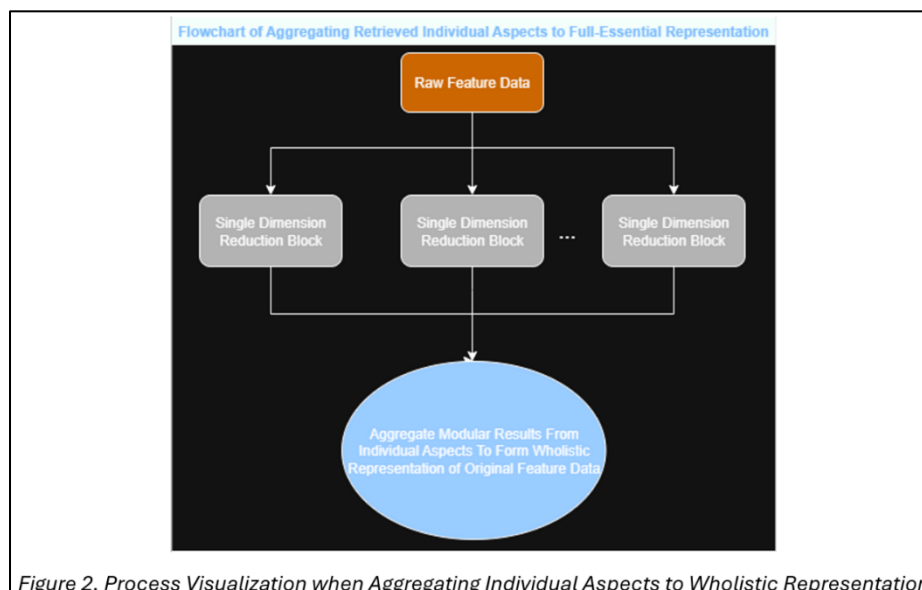
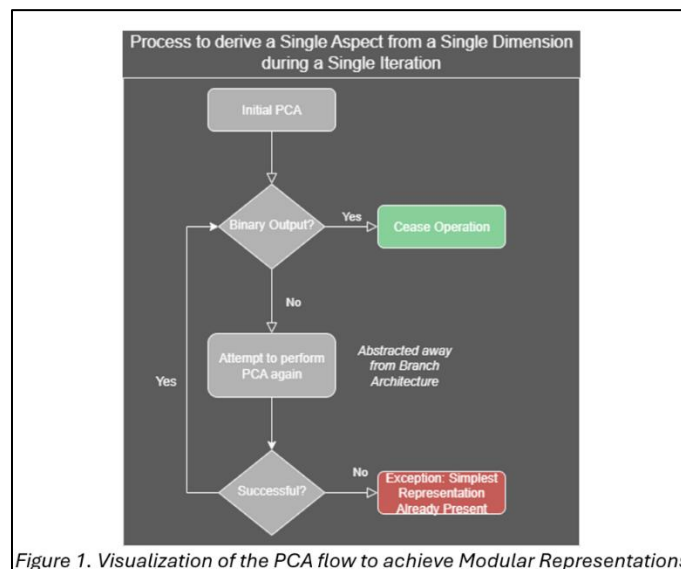
Furthermore, the results attained from stage 3 can also be passed through as input to an entire new procedure at stage 1. This is why it was emphasized that the constituent aspects are features onto themselves. For those familiar with it, this is similar to how the Convolutional Neural Network (CNN) architecture operates. Once the kernel extracts the features which it was designed for, pooling by some designated aggregation process takes place, in which such result is then passed to further blocks for further refinement. At the end of this procedure, there's usually some ranking process which incorporates DNNs with fully-connected layers; however, although important in other context, that specific portion is not substantial to this discussion. Although slightly different in purpose, the idea behind epochs is also similar to this endeavour as well. The difference here being, the dimensionality in which to analyze remains the same (training conducted multiple times using the same data).

Lastly, if the architecture in stage 3 designates an analytic operation which causes the variance to diverge upon multiple iterations through the procedure; obviously, responsive measures need to be in place to alleviate this in order to maintain a useful, tangible result. The process to deal with this differs depending on the compute resources available; however, the remedy usually involves gauging the limit of the interconnected outputs to inputs. This issue usually arises in optimization operations such as maximization and minimization. Specifically, it occurs in cases where the scale has the potential to change drastically during the foreseeable full operation of a procedure. In cases of averaging, the scale of results tends to remain relatively the same once normalized (which should be conducted regardless in order to retrieve a generalized interpretation that is relatively removed from outliers). Further cautionary notes will be provided in the “Pitfalls and Potential Remedies” section.

Opportunistic Restrictions

The main restriction inherent to the proposed methodology lies in the basic definition of what constitutes as a “dimension”. As dimensions need to be of tangible values once measured, the levels should consist of only positive values. Therefore, while there is no inherent limit on the maximal value for which it can take on, there is the absolute minimal limit of 0. As such, the lowest capacity in which information can occupy is the 0th dimension. Likewise, unlike the case with compounding dimensions, there is a distinct, physical limit to the extent a dimension can be decomposed. The term ‘physical’ in this sense is used to indicate a value that has a tangible correspondence. As a direct consequence of this hard minimum, any dimension which is classified as “multi-channel” can be broken down only as far as a binary option (“mono-channel”). Therefore, the case with the highest granularity is directly linked to the representation in which there is only one element. In other words, the contained information is capable of being adequately represented using a singular geometric point. It should be mentioned that in this context, the phrase “-channel” is meant to represent the format of elements used to express a unit of data. Hence, “multi-channel” refers to multiple elements each occupying their own dimension. An example is the RGB colour format. For this case, the RGB format is said to be a “3rd multi-channel” representation. “Mono-channel” representations are reserved for data which only require a single element (thus, a point) to be adequately represented. As the most basic type of binary representation is conveyed through an activated or deactivated response, a single element is sufficient to represent such cases. It’s simply a matter if the point is activated for the situation in which it represents. One scenario is in the monochromatic case, in which the state is represented by the percentage of the indicated colour occupying it. The endeavour to achieve binary representations stems from its ability for high traceability, as well as robustness. As mentioned previously, a modular representation is one of the primary goals of the proposed methodology. If higher dimensional representations are capable of being decomposed to multiple, binary constituent aspects, the stability of reasoning would be at its most efficient. This would ensure, given a specified scope of scale, defined causes correspond directly to calculated effects. In such a scenario, the probability of variance would be on a negligible basis, allowing the analyzer to report direct causality for all practical purposes. Due to this reason, for every problem with a tangible solution following analysis, the aim should be to attain a representation derived from terminally-branched decision trees consisting only of binary nodes. However, to achieve such a representation may require many iterative sessions, as well as produce an abundant number of inner branches. In other words, if a representation maintains a terminal branch with more than 2 nodes, there is a strong possibility that such a case can be

decomposed further into more granular, constituent aspects. This once again reinforces the point that the “scope of investigation” should be adhered to with the utmost of importance. Observations need to be scrutinized against a predefined scope in order to format an expression representing “binary” data from seemingly “non-binary” information. To achieve this; however, a number of assumptions need to be detailed and upheld, in addition to the resources required to compute such a representation. While the calculations themselves may not be intensive, the repetitive nature of the procedure may put intense stress on resources over time, which is a concern to keep in mind. A certain tolerance also needs to be agreed upon as a minute amount of information may be loss when striving for a binary representation. While this is an inherent consequence of PCA, the interpreter should still account for this, no matter how negligible its effects are. Figure 1 details the process of attempting to derive modular representations from a single feature. The particular branch design is abstracted away in that only the actions are focused on. As such, PCA should continue until the “most-approximate” modular representation on a specific feature is reached. Note, Figure 1 only highlights the utility of PCA. Thus, it doesn’t include branches or aggregation



functions, as those are further down the pipeline, which act on the notion of deriving various aspects from different perspectives.

Essentially, Figure 1 highlights the process to attain a modular representation of a single aspect. Figure 2 details the process of each modular representation attained, culminating together to devise a wholistic modular representation of the original data. Each modular representation of a particular aspect, derived from the original feature, is aggregated together with the other representations. The specific method of aggregation is dependent on the nature of data, as well as each aspect's influence on the original feature.

One of the main utilities of dimensional deconstruction is its ability to break data down into its constituent components, which can then be reconstructed into another format using the extracted information from the source (ex: text to image, image to sound). The (vanilla) 'Transformer' design architecture, which focuses on 'attention' achieves decomposition to a certain degree by focusing on aspects which relate to each other from within the relevant context. The primary issue with transformers is it still operates on information adhering to the specific dimension (i.e. cosine similarity as the loss metric dependent upon representative vectors). Therefore, it's dependent upon the granularity of scale for which their training data occupies. In addition, transformers (through many blocks) compound dimensions (semantic meaning, token composition, grammar, etc.) together to resemble a "natural-human" response (within a language scope). For this very reason, the modularity of transformers suffers, which make them susceptible to hallucination and retrieval errors, apart from the variance of training data. One of the applications of binary outcomes is to allow for discrete investigation and representation of data, which should in turn lead to less hallucinations (although, the response may become more rigid). Since binary responses provide "switch-like" interpretation, they are easy for models to traverse, as well as extract insights from. Due to the retraceable nature of such paths, determining foundational layers that conform to an overall pattern can be handled much more efficiently as compared to a multi-dimensional basis. It should be noted that LLMs lessen the hallucination by essentially reducing the variability among its parameters using the common "temperature" hyperparameter; however, this method can only alleviate so much due to the very nature of how the model was initially trained. In terms of analyzing data provided in one format which can then report insights using another format, multi-modal LLMs already accomplish this to a satisfactory degree. The issue here is the analysis of information is conducted only under the "language" dimension (from its namesake), thus restricting the depth in which data can be understood. If data can be broken down into their constituent components and ideally be represented as 'point-systems', regardless of the dimension, it should be universally representable, allowing it to be tangibly interpretable. In such situations, the scale or scope of the specific dimension would become negligible in understanding the data. Once a numerical representation is achieved, the understanding would be on an informational basis, which isn't restricted upon a subjective basis. This would further allow efficient processing of information, since it would be akin to simply thinking about the description of an object, rather than first thinking of the words which represent the description of such an object. For a simplistic analogy, one can think of the latent space within an Auto-encoder. Although, this is still working within a dimensional sense through the act of attempting to form an embedding; however, the idea is similar nonetheless. From this, it's clear that every design architecture has their pros and cons. It's up to the implementer to figure out how to capitalize on the pros and remedy the cons as much as possible. If an encoding of data is conducted, most likely it can be inferred to be statistical in its nature. As such, a certain tolerance needs to be accepted in order to confidently conduct analysis. This decision is the main trade-off, which provides an indication to the procedure's capacity in producing reliable insights.

Pitfalls and Potential Remedies

As mentioned previously, while there exist many benefits to using the proposed methodology; as with most architectures, it is not without its flaws. The majority of these were highlighted in the previous section, while the minor ones may be discoverable once implementation actively begins. Regardless, most minor problems are derived from the major issues detailed above. As such, potential remedies meant to alleviate some of the effects of these major issues will be presented hereafter. The majority of such remedies are presented in terms of a design perspective, rather than an implementation-based. This is done in that such cautionary notes may be applicable to a wider range of systems. Note, these are only guidelines which may allow for a more efficient implementation of dimensional manipulation. As such, the majority of details are left to the analyzer themselves, in their decision of how best to adhere to the core ideas highlighted throughout the paper. As every system has varying limitations, every implementation will also differ at least as much. With that said, there are essentially only a few cautions to keep track of. These were actually briefed previously, but will now be expanded upon, with the aim of offering coherent reasoning.

Firstly, the scope of investigation required to achieve a specified level of modularity should be defined before any implementation blocks are detailed. The pipeline from which dimensions are scrutinized are all built upon a particular scope. If a more granular result is required in respect to a relatively high dimensional data representation, the scope of investigation will likewise need to be increased. This in turn increases the procedural blocks required, which in turn puts a greater strain on compute resources. If resources are limited on a tasks' basis, then computation will usually compensate by simply allocating longer timespans (in accordance to the specific system's data loading and scheduling capabilities). As this problem isn't novel in any way, potential remedies have already been proposed; the majority of which include employing cloud-based resources or high-powered GPUs, TPUs, or NPUs. This is where balancing procedural capabilities, as well as the level of insights required should be highly discerned. While attaining the greatest level of modularity remains the aim of incorporating dimensional manipulation, it's not always feasible (given the immense amount of potential training data); therefore, not designated as a requirement. It's important to keep in mind the scope of investigation should only adhere to the required detail of insight set by the implementer for a particular system. Afterwards, another system may run through the results attained, which was designed under a higher scope of investigation. From this, an important observation can be detailed. Employing a pipeline-based approach of systems designed on varying scopes may prove more efficient than designating a highly-discerning scope upon the original, highly-dimensional data. This leads to the next guideline: procedural scheduling.

As its name suggests, procedural scheduling essentially defines the order and time in which data is processed during the analyzing phase. In the case of machine learning applications, this includes the training process. In an ideal scenario, both serialized and parallelized scheduling should be employed. Serialized scheduling allows for the pipeline-like design pattern to have a progressive effect on results, while parallelized scheduling allows for speed up in analyzation time in accordance to staged processing. Note, the largest advantage of this technique is seen in the processing a large amount of data (~1000s of records). In the case where there are few records (~100s of records), this technique may prove counter-productive, due to the extra resources required to implement it. While scheduling should be accounted for, the analyzer usually does not have to implement it from scratch. (Nvidia) GPUs have inherent parallelization on most relevant tasks, while CUDA kernels allow for modification on specific use-cases.

While the first two guidelines address the design of the methodology directly, the last cautionary note deals specifically with the interpretation of insights. At this point, a (single, in the simplest case) system is designed for a particular scope of investigation. As mentioned previously, insights derived from a particular system should always adhere to its designated scope. Hence, a set of insights are linked to a particular system. Since various systems can be chained together to inspect different aspects, it's paramount that the analyzer is still able to discern insights based on a particular scope of investigation. Thus, within the entire procedure, there may exist multiple systems, each employing their own methodology designated upon a specified scope. As such, there exists a class of "sub-insights", derived from a particular scope, for which may be subject to further manipulated. More concretely, the insights derived can be manipulated across systems, leading to 'inter-system' dependence. For this reason, it's advisable that a timestamp be linked (ideally concatenated during implementation) with each insight by default. This not only allows for better analysis through surveying the pool of data, but also heightens the capacity to track results once the systematic map becomes increasingly complex. While this may be done implicitly as inferred from the analysis logs, it should be explicitly stated, especially for this case. Afterall, analysis requires not only the capacity of computation systems, but also the knowledge of the analyzer to comprehend derived results. This would further prove beneficial in the case where results previously attained are used as input data within another analytic procedure. Note, this technique should aid in any temporal-based analysis, not just dimensional manipulation. Recall, the source of insights is derived from the particular system within a procedure. Therefore, the possibility may arrive where further branches are required in the decision process, motivated by further appointed requirements. As such, the concatenated timestamps would act as a type of memory or recollection medium, which can be incorporated into the present analysis. Furthermore, it allows for the analyzer to gain some kind of situational awareness, in that to gauge the timespan a set of data was collected, so its relative quality may be discerned when taken into consideration. Afterall, as human-based decisions are subject to change, insights should have a tracking mechanism along the temporal dimension. In this way, critical insights can be derived using the most updated and relevant data available.

Proof of Concept (Possible Implementation)

Highlights

- Full scripts, notebooks and instructions are available at:
https://github.com/Kureishi/Dimensional_Manipulation_Public
- As PCA is likely to be the most well-known applicative process of dimensional manipulation (specifically, reduction), it was chosen to demonstrate the main concepts of the methodology
- A naïve implementation of the PCA algorithm was employed as to keep background and domain expertise to a minimum. In this way, the implementation would be able to act as a foundation to a wider range of applications.
- The implementation presented is only one of many. As previously mentioned, the paper aims to provide insight into the contributions of dimensional manipulation; it's a

methodology, not an algorithm. Therefore, the entire process is intended to be the main focus, rather than the specific code blocks which implement it.

- Only the main points will be provided (with snapshots). For further details, refer to the GitHub repository.

Main Class

- For full disclosure, this class was partially coded with the use of AI tools (Windsurf, 2025). To reiterate, this was a purposeful step as to make the implementation relatively simple to understand and follow. Since AI generally conducts coding tasks on how others would implement the process (accumulated through its main training material), it was deemed beneficial to have it code the backbone of a naïve (vanilla) PCA implementation rather than researching a specific implementation.
- The *fit* method of the class provides its main definition. Basic functionality includes calculating the covariance matrix among all features associated with the input data. It then derives the eigenvalues (variance contribution of each principal component to the data) and eigenvectors (reduced dimensional component of each feature) using eigenvalue decomposition.
- The *transform* method actually performs the projection of the input data onto the deduced principal components, thereby reducing its representative dimensions
- The principal components are ranked based on their eigenvalues. As such, the components which appear early in the dictionary explain more of the input data's variance. The number of components (dimensions) to retain are indicated by the *n_components* property.
- A general property of eigenvectors is they are normalized to a unit length. Therefore, the sum of squares for all elements pertaining to a particular feature of all principal components is approximately equal to 1. The direction of each eigenvector points in the direction of their relative variance. Similar direction indicates a similar 'type' of explainability.
- Eigenvectors with the highest associated eigenvalues point in the direction of the most variation of data (which explains the 'behaviour' of the data). When data is projected onto these components, it's effectively compressed/reduced to a lower dimension, while maintaining a reasonable amount of its explainability. As the data retains more dimensions, it should be clear of how it would demonstrate more of its original explainability.
- The *conditional constructor* is employed to change the initialization depending on which arguments are passed in. In this way, the initialization of the class object can be done from original data or by using pre-defined "components (stored in *pca_dict*) and explained variance (stored in *explained_variance*)" pair. This is particularly useful during the 'pipeline' stage of chaining different PCA components together to derive various insights.

```

# Attain corresponding eigenvalues and eigenvectors based on covariance matrix of data (X)
def fit(self, X):
    # Center the data (so derived relationships are relevant among features)
    self.mean_ = np.mean(X, axis=0)
    X_centered = X - self.mean_

    # Calculate covariance matrix
    cov_matrix = np.cov(X_centered.T)

    # Perform eigenvalue decomposition
    eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

    # Sort eigenvalues and eigenvectors in descending order
    sorted_indices = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[sorted_indices]
    eigenvectors = eigenvectors[:, sorted_indices]

    # Store explained variance
    self.explained_variance_ = eigenvalues

    # Select top n_components
    if self.n_components is None:
        self.n_components = X.shape[1]

    # Set components to the allocated number of eigenvectors to retain
    self.components_ = eigenvectors[:, :self.n_components]

    return self

```

Figure 3. fit data to retrieve PCA components

```

def __init__(self, n_components=None, pca_dict=None, explained_variance=None):
    self.mean_ = None
    # if only n_components are passed -> initialize PCA from scratch
    if n_components:
        self.n_components = n_components
        self.components_ = None
        self.explained_variance_ = None
    # if only pca_dict and explained_variance_ are passed -> initialize per PCA information
    elif pca_dict and explained_variance_:
        self.pca_dict = pca_dict
        self.components_ = pca_dict
        self.explained_variance_ = explained_variance_
    else:
        raise ValueError("Either n_components or both pca_dict and explained_variance_ must be provided")

```

Figure 4. conditional constructor

Procedural Functions

- These functions are utilized to derive insights through each stage of a single analytical process. These include generating the principal components and transforming the relative data upon them. The specific implementation of the modified components is also detailed in such functions, in that slight modifications may be conducted to extract various attributes. The aggregation stage is also implemented here.
- It may be important to note that the principal components were specifically initialized through a dictionary construct. In this way, the construct is accessible when the dimensions are recursively reduced to a modular-binary format (concept highlighted previously). For this, the key of such dictionary indicates the dimensions (thus, the components) it reduces the data on. For example, 'pca_4' reduces the data to 4 dimensions. This was only done to demonstrate the possibility and potential implementation of the process in which dimensions; thus, representative data, may be iteratively reduced.

- The functions which derive insights targeted on a particular attribute are also defined in this section. As this is highly dependent on the nature of said attribute, the functions are likewise nuanced in their implementations.
- In addition, the aggregation function is also dependent on its particular formulation. For this particular case, the average of two sets of sub-components were conducted.

General Flow

- 1) Original Data Defined and Indexed
- 2) Center the Data followed by Calculating Covariance Matrix
- 3) Derive Eigenvalues & Eigenvectors using Eigen Decomposition
- 4) Extract Principal Components based on acceptable Variance
- 5) Transform Data through PCA by fitting it on Components
- 6) Original Data Reduced to its Representative Dimensions

Specific Designated Implementation Procedure

- i. Utilize eigenvalues to dictate the flow of dimensionality reduction
- ii. Attempt to reduce until modular form (exemplified by *pca_1*), or to the point where explainability arrives at the predefined threshold
- iii. Explainability of representative dimensions dictated mainly by utilizing the *explained_variance_ratio_* property to provide insight into feature contribution
- iv. Add components of corresponding eigenvalues until the sum of such values achieves the accepted variance ratio threshold

Examples of PCA Function Manipulation

- Limiting representative dimensions to a certain variance which allows for the generalization to be set (exemplified in demonstration)
- Prioritizing certain features more than others (utilize domain expertise)
- Dissecting only certain features while neglecting others to allow for greater efficiency in pattern recognition
- Combining and decomposing derived/engineered features from the original dataset

Notes on Overall Implementation

One main application of this specific approach is image segmentation. Specifically, identifying different aspects (potentially, engineered features) of a particular image. As mentioned previously, this is similar to the case of CNNs. The point of this sample implementation however (and the entire paper, for that matter), is to highlight the flexibility and adaptability of such techniques in their applicability to other domains.

Since the functions are tuned for their relative purpose, it may not be directly reusable for another use-case. Therefore, it was deemed inefficient to format the functions into a library, which may later be used by someone else. It's much more beneficial to detail the process and techniques rather than specific code when the capacity to generalize is a priority. Since a great deal of explanation is included within the associated notebooks and scripts, it will therefore be omitted here.

Since this sample implementation was conducted for demonstration purposes only, a randomly-generated, numeric data set was employed. However, this data can just as easily be interchanged using another numeric dataset (or data represented numerically). For example, the MNIST dataset (images of numeric digits) can have representations which measures different aspects (such as size, shading, value indication, etc.). Through utilizing different PCA procedures tuned to a specific aspect, insights of greater detail and relevance may be produced.

It should be noted that some features don't explain any tangible variance of the data. Such is the case of the *pca_1* variable, defined on 2 components. Specifically, 2 components with 1 element each (2 dimensions $\rightarrow$ 1 dimension). As mentioned previously, the more dimensions employed increases the capacity for variation in representations. As such, the opposite can also be inferred. The fewer the dimensions utilized, the larger the decrease in capacity for representation. However, this is to be expected as the trade-off from generalization to representation. Thus, it can be further inferred that at some point during the reduction, the variance of a dimension would be fully explained by one aspect rather than another (to a measurable extent). In other words, one representative aspect would have "greater importance" than another. Through this measurement, filtering through dropouts of certain aspects may be possible, which can further improve the efficiency of pattern recognition and inference.

The pipeline structure and engineering of different aspects from within each dimension are by their nature fully customizable. This process entails a great deal of trial-and-error in determining which configuration achieves the expected result within a defined threshold. A simple system was implemented for a proof-of-concept (averaging two modifications on the original derived components); however, this can become much more complicated as the solution dictates. As long as the aggregation operation is tangible on the present data types, the pipeline can be formatted as seen fit by the user. For specific details on how this may be possible, refer to the associated notebook or script.

Future Implications

The possibilities for applications of dimensional manipulation are realistically endless, as was known for a long time. This is due to the scale in which perception and processing is designated on for a particular system. It depends on the applicator, processor, and receiver. Once the scope of which to conduct investigation is defined, all insights are derived accordingly. Therefore, the foundation of any dimensional manipulation application should always adhere to the scope for which it is defined. The higher the scope, the more detailed and deeper insights will be reported upon. However, these computations will likewise utilize an intense number of resources. In contrast, the opposite reasoning can mainly be inferred for a lower scope.

The domains in which dimensional manipulation may be applied to are similarly vast. It is not only traditional STEM applications which benefits from these techniques; they benefit any domain which processes data. Hence, any modern domain would therefore be applicable. The only difference are the methods employed to do so may not be as straight forward. As the dimensions get larger, the representations are likewise increasingly complex. This goes back to the idea of signifying complexity by the number of modular representations taken for a sample of data. That is to say, the more straight-forward the dataset is, the less dimensions would be required on a relative scale. For instance, if numbers are represented through different modalities, such as the case of audio and visuals. Specifically, the number 1 can be represented by either drawing a '1' or speaking the word "one". As a simple statistic (that should be readily verifiable), the variations in which the number '1' is written tends to have more variation when compared to how it is spoken. However, if the demographic is taken through a broader perspective, this observation starts to shift towards an alternate conclusion (mainly due to accents and linguistic restrictions). Therefore, a slight change in the aspect on which a dimension is measured can lead to high variability in consequences.

With that said, the process of dimensional manipulation as a whole can be applied to just about any application which requires the cause-to-effect relation to be explicitly defined. Hence, it may be a classic example of determining factors of market inclination or a subtle case of deciding whether to use chocolate 'chips' or 'chunks' in a batch of cookies. The encompassing goal of this paper is to exhibit possible ways in which a different analytical strategy may lead to deeper results, for which was not explored previously. Thus, these techniques have tangible effects on any analytical procedure.

In solving problems, a procedure commonly utilized is breaking it down into simpler tasks, for which each such solution is then aggregated into an encompassing solution. This is the ultimate goal of dimensional manipulation techniques. For any analytical problem (in most cases), it may be broken into simpler problems (aspects of a dimension). These simpler problems can potentially further be broken into smaller parts (aspects become a dimension onto themselves), and the process continues. The only exception being, as mentioned before, is if the original problem is already in an "atomic" state, in which it can't be broken down further (binary-modular form). Through this, one can visualize how they may implement such techniques which are not limited to classical domains.

References

Bergmann, D. (2025, January 28). What is latent space? *IBM Think*. Retrieved May 15, 2025, from <https://www.ibm.com/think/topics/latent-space>

Windsurf. (2025). SWE-1 [Large language model]. <https://windsurf.com/editor>